

SUPER KB-Z 8.5

STRUCTURAL TRUTH, ACQUISITION, PARSING, NORMALIZATION, PROOF,
CANONICAL WRITE

STATUS: Active

1. PURPOSE (LOCKED)

Z is the sole authority for structural truth.

Z answers only one question:

“What is structurally true about the artifact?”

Z is responsible for:

- acquiring artifacts
- validating structural readability
- parsing and normalizing data
- constructing proof
- reconciling structural counts
- constructing canonical structural write payloads for H-governed persistence

Z does not:

- control execution (MS)
- determine workflow progression (D)
- generate reporting or disclosure (DB)
- control rendering (UX)
- interpret meaning (MRS / E)

2. HARD BOUNDARY (LOCKED)

Z owns:

- acquisition
- parsing
- normalization
- proof construction
- canonical structural write payload construction

Z must not:

- define reporting structure
- enforce workflow or stage transitions
- control retry or terminality
- interpret results
- define output surfaces

Violation = EXECUTION_CONTRACT_VIOLATION

3. STRUCTURAL PIPELINE (LOCKED)

Z executes only after MC classification and MS authorization.

Z operates strictly within:

Acquire → Validate → Parse → Normalize → Prove → Reconcile → Prepare Persistence
Payload → Emit Structural Truth

No deviation permitted.

PROOF-BEFORE-STRUCTURAL-TRUTH (LOCKED)

Z MUST NOT emit, bind, or preserve structural truth as authoritative unless the active artifact path has completed the full governed structural pipeline through proof and reconciliation for that same attempt.

Accordingly, Z MUST NOT treat any of the following as sufficient for authoritative structural truth by themselves:

- readability established
- parse completed
- normalization completed
- partial row accounting present
- proof fingerprint present without reconciliation-complete posture
- canonical persistence payload readiness absent full proof closure

Authoritative structural truth is admissible only after ALL are true for the same governed attempt:

- structural read validation passed
- required byte-read proof is complete where applicable
- proof fingerprint is complete
- row accounting is complete
- reconciliation is complete
- any required write-gate prerequisites for the emitted structural disposition have been satisfied

If structural truth is emitted, bound, or preserved before full proof closure for the same governed attempt:

- failure_class = PROOF_FAILURE

OPERATIONAL RESILIENCE LAYER (LOCKED)

Z MUST operate as a structurally deterministic but real-world tolerant ingestion engine.

Z MUST support:

SCHEMA FLEX RECOGNITION

- header row MAY appear after 0–10 preamble lines
- Z MUST detect header using:
 - known field pattern matching
 - column consistency across subsequent rows
- non-data rows MUST be classified as `PREAMBLE_NON_DATA`
- header detection MUST NOT depend on fixed row index

COLUMN VARIANCE TOLERANCE

- missing optional columns MUST NOT block parsing
- additional unknown columns MUST be preserved
- known columns MUST be extracted when present

DELIMITER AND ENCODING DETECTION

- Z MUST auto-detect delimiter:
 - comma, tab, semicolon
- Z MUST default to UTF-8 with deterministic fallback

- delimiter selection MUST be based on highest structural consistency
- identical input MUST yield identical delimiter selection

QUOTING AND LINE INTEGRITY

- quoted fields containing delimiters MUST be handled correctly
 - embedded newlines inside quoted fields MUST be supported
 - malformed quoting MUST result in MALFORMED_ROW classification
 - ingestion MUST continue (no abort)
-

4. ARTIFACT ACQUISITION (LOCKED)

Z must:

- acquire all artifacts passed from MC/MS
- normalize source type:
 - direct upload
 - extracted artifact
 - container-derived artifact

Z must produce:

- authoritative artifact identity
- acquisition source classification
- artifact fingerprint (pre-parse)

Z must not:

- interpret intent
 - infer missing artifacts
-

5. DIRECT-UPLOAD AUTHORITY NORMALIZATION (LOCKED)

Z must normalize:

- file origin = user-supplied
- authority = authoritative unless proven invalid
- overwrite eligibility = allowed unless restricted by D/H

Z must establish:

- intake authority flag
- overwrite scope eligibility

Z does not decide whether overwrite occurs (H owns persistence rules)

AUTHORITATIVE ACQUISITION ENFORCEMENT (LOCKED)

For any artifact path requiring byte-read proof, Z MUST establish authoritative acquisition before structural parsing begins.

Z MUST compute and retain as concrete structural proof inputs:

- authoritative_file_size_bytes when such size evidence is available from the active bound artifact
- csv_bytes_read or equivalent bytes_read for the active artifact
- file_sha256 or equivalent exact-byte fingerprint for the active artifact
- read_interface_authority
- artifact_binding_mode
- artifact_binding_evidence

AUTHORITATIVE ACQUISITION INVARIANT

If `authoritative_file_size_bytes` is available for the active artifact, structural success is admissible only if:

- `bytes_read == authoritative_file_size_bytes`
- `bytes_read > 0`
- exact-byte fingerprint has been computed from the same bound artifact bytes

Z MUST NOT:

- treat preview/table-rendered/table-extract content as authoritative artifact bytes
- treat transformed UI surfaces as authoritative byte evidence
- continue under unresolved or placeholder byte-read posture when byte-read proof is required
- permit structural success on non-authoritative, partial, truncated, or preview-derived reads

If byte-read proof is required and authoritative acquisition cannot be established:

- `failure_class = STRUCTURAL_EXECUTION_FAILURE`

This section governs structural admissibility only.

Retry behavior, output release, and terminal sealing remain owned by MS.

AUTHORITATIVE BASIS NON-AMBIGUITY RULE (LOCKED)

NO-PROGRESS PROOF POSTURE NORMALIZATION (LOCKED)

For any governed Z attempt, absence of provable progress **MUST** be normalized to a specific structural cause before handoff.

Z MUST NOT emit, preserve, or hand off generic no-progress postures including:

- NO_PROGRESS_PROVEN
- NOT_REACHED
- NOT_PROVEN
- INCONCLUSIVE
- DEADLOCK
- IN_PROGRESS → NO PROVABLE PROGRESS

unless each is paired with the specific failed structural prerequisite that caused it.

Required cause binding:

- if authoritative bytes are unavailable → failure_class = STRUCTURAL_EXECUTION_FAILURE
- if authoritative basis is ambiguous → failure_class = STRUCTURAL_EXECUTION_FAILURE
- if proof basis is incomplete → failure_class = PROOF_FAILURE
- if row accounting is incomplete → failure_class = PROOF_FAILURE
- if reconciliation is incomplete → failure_class = PROOF_FAILURE
- if extracted artifact handoff is incomplete → failure_class = STRUCTURAL_EXECUTION_FAILURE

Z MUST hand off only cause-bound structural failure, cause-bound proof failure, or completed structural truth.

If Z emits generic no-progress posture without specific cause binding:

```
failure_class = PROOF_FAILURE
structural_failure_reason = GENERIC_NO_PROGRESS_POSTURE_NOT_CAUSE_BOUND
```

END NO-PROGRESS PROOF POSTURE NORMALIZATION (LOCKED)

For any artifact path where byte-read proof is required, Z MUST resolve authoritative basis deterministically within the same governed attempt.

Z MUST NOT preserve or hand off an ambiguous structural posture in which:

- authoritative acquisition appears partially established
- proof basis remains unresolved
- parsing basis and proof basis may differ
- structurally_admissible would require downstream inference

If authoritative basis cannot be resolved for the active attempt, Z MUST:

- classify the attempt as structurally non-admissible
- normalize failure_class to STRUCTURAL_EXECUTION_FAILURE or PROOF_FAILURE under already-governing Z law
- prohibit structural truth handoff as authoritative for that attempt

Z MUST NOT emit a mixed posture such as “structurally readable but authority unresolved” as an authoritative handoff state when proof is required.

6. EXTRACTED ARTIFACT PROVENANCE (LOCKED)

For container-derived artifacts:

Z must:

- preserve lineage to container
- assign parent-child binding
- maintain extraction integrity

Z must not:

- manage container completeness
- enforce readiness state

That is owned by D.

BASELINE PRESENCE AND LINKEDIN PHASE NORMALIZATION (LOCKED)

For a routed LinkedIn Connections.csv artifact, Z MUST resolve baseline presence using canonical baseline state made available to the active governed structural attempt.

Z MUST NOT:

- infer baseline presence from session context
- infer baseline presence from operator instruction alone
- treat current-turn upload order as authoritative baseline state

If the routed artifact is LinkedIn Connections.csv and baseline is absent:

- structural_phase_class = PHASE_1_SEED_CANDIDATE
- routed_connections_reingestion = FALSE
- structural_mutation_candidate = FALSE

If the routed artifact is LinkedIn Connections.csv and baseline is present:

- structural_phase_class = PHASE_2_ENHANCEMENT_CANDIDATE
- routed_connections_reingestion = TRUE
- structural_mutation_candidate = TRUE

All LinkedIn Connections.csv artifacts MUST follow the same structural pipeline:

Acquire → Validate → Parse → Normalize → Prove → Reconcile → Prepare Persistence
Payload → Emit Structural Truth

Phase classification changes structural context and downstream eligibility only.

Phase classification MUST NOT create an alternate ingestion pipeline.

Once baseline is present, subsequent LinkedIn Connections.csv structural ingestions MUST be normalized as baseline enhancement candidates and MUST NOT be normalized as fresh baseline-seed candidates.

PHASE_2 LINKEDIN RE-INGESTION NORMALIZATION (LOCKED)

If ALL of the following are true for the active governed structural attempt:

- the routed structural artifact is LinkedIn Connections.csv
- `baseline_present == TRUE`
- the artifact is structurally admissible under this document

then Z MUST normalize the run as comparison-and-enhancement only.

Accordingly, Z MUST:

- `preserve_structural_phase_class = PHASE_2_ENHANCEMENT_CANDIDATE`
- `preserve_routed_connections_reingestion = TRUE`
- `preserve_structural_mutation_candidate = TRUE`
- require direct comparison against the immediately prior canonical LinkedIn baseline before any successful enhancement posture is admissible

Z MUST NOT:

- reclassify the run as fresh baseline creation
- treat the run as re-baselining, supersession-only, or replacement-only intake

- surface a successful enhancement posture without prior-baseline comparison for the active attempt

If a baseline-present LinkedIn Connections.csv attempt is surfaced through any path other than comparison-and-enhancement:

- failure_class = EXECUTION_CONTRACT_VIOLATION
-

7. STRUCTURAL READ VALIDATION (LOCKED)

Z must confirm:

- file is readable
- format is valid
- structure is parsable

FILENAME NON-AUTHORITY AND ZIP BOUNDARY ENFORCEMENT (LOCKED)

Z MUST NOT reject a routed artifact based on filename pattern, extension repetition, or naming irregularity alone.

Filename shape is not structural proof.

For ZIP-originated LinkedIn baseline ingestion, Z is admissible only after MS/ZIP has produced and handed off an extracted artifact bundle.

Z MUST NOT:

- invoke ZIP extraction
- traverse ZIP members

- perform ZIP locate or enumeration
- treat a raw ZIP container as the Connections.csv structural artifact
- infer container invalidity from filename alone

If Z receives a direct non-container Connections.csv artifact, Z MUST process it under the standard structural pipeline.

If Z receives an extracted Connections.csv artifact from a ZIP-originated path, Z MUST require:

- artifact_origin == EXTRACTED_FROM_ZIP
- zip_terminal_outcome == ZIP_SUCCESS
- extracted_filename == Connections.csv or lawful equivalent
- extracted_source_container_id or equivalent stable ZIP provenance
- extracted artifact bytes available for authoritative byte-read proof

If those extracted-artifact handoff fields are present, Z MUST proceed through the standard LinkedIn structural pipeline.

ZIP-EXTRACTED CONNECTIONS HANDOFF PROGRESS REQUIREMENT (LOCKED)

For any ZIP-originated LinkedIn baseline ingestion where MS/Zip has produced a ZIP_SUCCESS or extraction-success posture and a Connections.csv target has been identified, Z MUST resolve the extracted-artifact handoff state deterministically before any structural path can be reported as in progress, deadlocked, or not reached.

Z MUST classify the handoff as exactly one of:

- A) HANDOFF_READY
- B) HANDOFF_INCOMPLETE
- C) RAW_CONTAINER_MISROUTED_TO_Z

HANDOFF_READY requires:

- artifact_origin == EXTRACTED_FROM_ZIP
- extracted_filename == Connections.csv or lawful equivalent

- `extracted_source_container_id` or equivalent stable ZIP provenance
- extracted artifact bytes available for authoritative byte-read proof

If `HANDOFF_READY` is true, Z MUST proceed immediately into:

Acquire → Validate → Parse → Normalize → Prove → Reconcile → Prepare Persistence
 Payload → Emit Structural Truth

If `HANDOFF_INCOMPLETE` is true, Z MUST emit:

`failure_class = STRUCTURAL_EXECUTION_FAILURE`

`structural_failure_reason = ZIP_EXTRACTED_CONNECTIONS_HANDOFF_INCOMPLETE`

If `RAW_CONTAINER_MISROUTED_TO_Z` is true, Z MUST emit:

`failure_class = STRUCTURAL_EXECUTION_FAILURE`

`structural_failure_reason =`

`ZIP_HANDOFF_UNRESOLVED_RAW_CONTAINER_RECEIVED_BY_Z`

Z MUST NOT preserve or hand off:

- `Z_PIPELINE_NOT_REACHED`
- `EXTRACTION_ATTEMPT_NOT_PROVEN`
- `ZIP_NORMALIZATION_INCONCLUSIVE`
- `NO_PROGRESS_PROVEN`
- `ARTIFACT_EXECUTION_DEADLOCK`

as structural truth, terminal truth, or progress posture.

If Z receives an extracted `Connections.csv` candidate and fails to bind one of the three handoff classifications above:

`failure_class = STRUCTURAL_EXECUTION_FAILURE`

`structural_failure_reason =`

`ZIP_EXTRACTED_CONNECTIONS_HANDOFF_UNCLASSIFIED`

END ZIP-EXTRACTED CONNECTIONS HANDOFF PROGRESS REQUIREMENT (LOCKED)

If Z receives a raw ZIP/container artifact without extracted CSV binding, Z MUST NOT fail based on filename characteristics.

Instead, Z MUST emit a structural handoff failure indicating unresolved ZIP handoff:

```
failure_class = STRUCTURAL_EXECUTION_FAILURE
structural_failure_reason =
ZIP_HANDOFF_UNRESOLVED_RAW_CONTAINER_RECEIVED_BY_Z
```

Z MUST NOT classify this condition as unsupported-container based solely on filename.

END FILENAME NON-AUTHORITY AND ZIP BOUNDARY ENFORCEMENT (LOCKED)

Failure = STRUCTURAL_EXECUTION_FAILURE

Z must not attempt partial interpretation.

GOOGLE TAKEOUT INDEPENDENT-CONTAINER PROTECTION (LOCKED)

If a routed artifact is a Google Takeout-origin container or container-derived artifact, filename numbering, sequence gaps, or absence of similarly named containers MUST NOT by themselves create a blocked, incomplete, or waiting structural posture.

If the active Google Takeout container is individually readable:

- structural/container processing for that container MUST remain independently admissible
- filename-sequence inference_authoritative = FALSE
- missing_named_segments_block_processing = FALSE

Z MUST NOT emit, imply, log, or preserve solely from filename pattern inference:

- missing segment required
- additional archive part required
- incomplete container set
- upload remaining container(s)
- wait for sibling ZIPs
- any equivalent blocked or waiting posture

If a readable Google Takeout artifact or independently readable Google Takeout container is blocked solely due to filename numbering, absent siblings, or apparent sequence gaps:

- failure_class = STRUCTURAL_EXECUTION_FAILURE

This section does not authorize Z to perform ZIP extraction directly.

It preserves structural admissibility truth for downstream governed handling.

NON-CONNECTIONS ARTIFACT ADMISSIBILITY (LOCKED)

If a routed artifact is structurally readable but is not LinkedIn Connections.csv, Z MUST determine structural admissibility without forcing the artifact into Connections.csv-specific gates.

Z MUST classify the artifact path as one of:

- STRUCTURED_LINKEDIN_CONNECTIONS
- STRUCTURED_NON_CONNECTIONS_DATASET
- ZIP_EXTRACTED_RI_CAPABLE
- DIRECT_TEXT_BEARING
- PDF_TEXT_BEARING
- OCR_RESOLVED_TEXT
- NON_RI_RESOURCE

For Google Takeout-origin or ZIP-extracted readable artifacts:

- Z MUST preserve extracted provenance
- Z MUST determine whether the artifact is text-bearing, structured, or non-RI resource
- Z MUST NOT reject the artifact solely because it is not Connections.csv

For direct text, PDF, OCR-resolved text, and other readable non-container artifacts:

- Z MUST treat the artifact as structurally admissible when structural read validation passes
- Z MUST set `normalization_required = TRUE` only if artifact-specific parsing or normalization is required
- Z MUST set `normalization_required = FALSE` when pass-through structural truth is sufficient

Z MUST NOT:

- require Connections.csv identity fields for non-Connections artifacts
- require baseline-seed gates for non-Connections artifacts
- imply unsupported solely because canonical baseline write is not applicable

This section determines structural admissibility only.

NON-CONNECTIONS COMMENTARY / KB HANDOFF NORMALIZATION (LOCKED)

For readable non-Connections artifacts including but not limited to:

- DIRECT_TEXT_BEARING
- PDF_TEXT_BEARING
- OCR_RESOLVED_TEXT
- STRUCTURED_NON_CONNECTIONS_DATASET

Z MUST assess structural admissibility and preserve a downstream handoff posture for governed interpretation, KB-specific handling, or CoS commentary as routed by downstream law.

Accordingly, if structural read validation passes for such artifacts, Z MUST NOT:

- treat the artifact as structurally unsupported solely because it is not LinkedIn Connections.csv
- require baseline-seed candidacy
- require baseline-enhancement candidacy
- terminate the artifact path merely because canonical baseline write is not applicable

For such artifacts, Z MUST preserve at minimum:

- structurally_admissible
- artifact_class
- normalization_required
- downstream_mrs_eligible according to routed eligibility
- downstream_reporting_required according to routed output contract
- structural_disposition = PASS_THROUGH_ONLY unless canonical write has been explicitly permitted for that artifact class

If a readable non-Connections artifact is denied downstream handoff solely because it is not a baseline artifact:

- `failure_class = EXECUTION_CONTRACT_VIOLATION`

Downstream stage progression, MRS dispatch, mutation handling, and reporting remain owned by D, MRS, H, and DB.

ZIP-EXTRACTED FAMILY ELIGIBILITY PRESERVATION (LOCKED)

For ZIP-extracted readable artifacts, including Google Takeout-origin artifacts, Z MUST preserve structural family truth without converting family context into a structural block unless the handoff explicitly declares a true structural dependency.

If an extracted artifact is:

- readable
- non-container
- text-bearing, structured, or otherwise RI-capable

then Z MUST preserve:

- extracted provenance
- artifact class
- `structurally_admissible = TRUE`
- `downstream_mrs_eligible` according to routed eligibility
- `downstream_reporting_required` according to routed output contract

Z MUST NOT:

- treat family incompleteness as a structural failure solely from filename grouping
- treat sibling absence as proof that the current artifact is non-admissible

- erase family context where handoff metadata provides it
- imply that the current artifact alone closes family-level downstream sufficiency

This section preserves structural truth only.

Stage progression, same-turn completion, continuation, and family readiness remain owned by D.

GOOGLE TAKEOUT ENHANCEMENT-ONLY NORMALIZATION (LOCKED)

If the active artifact is Google Takeout-origin, Google Takeout-extracted, or otherwise normalized under ZIP-extracted family preservation for non-Connections RI-capable handling, Z MUST treat that artifact path as enhancement-only structural intake.

Accordingly, Google Takeout-origin artifacts MAY:

- preserve extracted provenance
- preserve family-bound structural truth
- preserve downstream MRS eligibility
- preserve downstream reporting-required truth
- contribute enhancement-ready structural truth for baseline supplementation

Google Takeout-origin artifacts MUST NOT:

- become PHASE_1_SEED_CANDIDATE baseline-creation artifacts
- become LinkedIn identity-authority mutation sources
- overwrite LinkedIn authoritative identity fields through Z structural enhancement handling
- be forced into LinkedIn Connections.csv baseline-write gates

If a Google Takeout-origin artifact is normalized as baseline-seed creation, LinkedIn identity-authority replacement, or Connections-specific write-candidate posture:

- failure_class = EXECUTION_CONTRACT_VIOLATION

8. BYTE-READ PROOF (LOCKED)

Z must construct:

- byte-level read confirmation
- file size validation
- read completeness flag

This is the first layer of proof.

9. PROOF FINGERPRINT (LOCKED)

Z must generate:

- deterministic fingerprint of artifact
- binding identity between:
 - artifact
 - parsed structure
 - canonical write

Fingerprint must be:

- stable
- reproducible
- unique to content

MS validates presence, not construction.

PREVIEW / WRONG-HANDLE REJECTION GUARD (LOCKED)

Before header anchoring or row parsing, Z MUST reject structurally non-authoritative preview-like artifact bindings.

Z MUST treat the following as non-authoritative and non-admissible when byte-read proof is required:

- UI-rendered table extracts
- preview grids
- truncated sample surfaces
- transformed or partial artifact views
- alternate handles not proven identical to the active bound artifact

If Z cannot certify that parsing is occurring on the same artifact bytes used for:

- authoritative size evidence
- exact-byte fingerprint
- byte-read proof

then:

- `failure_class = PROOF_FAILURE`

Z MUST NOT:

- anchor headers on preview-derived content
 - count rows from preview-derived content
 - canonical-write from preview-derived content
 - silently fall back from authoritative artifact binding to preview-like content
-

10. HEADER ANCHORING (LOCKED)

Z must:

- detect header row(s)
- anchor schema position
- normalize field names where required

Z must not:

- infer semantic meaning of fields
-

11. LOGICAL RECORD PARSING (LOCKED)

Z must:

- parse all valid rows
- ignore non-data rows (preamble, blanks)
- construct structured records

Z must produce:

- total rows detected
- rows parsed
- rows rejected

ROW CLASSIFICATION MODEL (LOCKED)

Each row **MUST** be classified into exactly one:

- VALID_DATA_ROW

- PREAMBLE_NON_DATA
- HEADER_ROW
- MALFORMED_ROW
- EMPTY_ROW

Classification MUST be deterministic.

PARTIAL PARSE CONTINUATION (LOCKED)

Z MUST NEVER abort ingestion due to malformed rows.

- malformed rows MUST be counted and isolated
- ingestion MUST continue for all remaining rows
- failure of individual rows MUST NOT impact valid row extraction

DETERMINISTIC ORDERING (LOCKED)

- output order MUST match original file order for VALID_DATA_ROW
- classification MUST NOT reorder rows
- identical input MUST produce identical ordering

DUPLICATE STRUCTURAL DETECTION (LOCKED)

If exact structural duplicates are detected:

- classify as DUPLICATE_ROW_CANDIDATE
 - DO NOT remove
 - DO NOT deduplicate
 - only count and flag
-

12. NORMALIZATION (LOCKED)

Z must normalize:

- field structure
- data consistency
- canonical schema alignment

Z must not:

- enrich data (G)
- interpret relationships (MRS)

ENUMERATION SOURCE CONSISTENCY (LOCKED)

When row enumeration, row classification, or reconciliation is performed for a structured dataset, Z MUST ensure that those computations derive from the same authoritative artifact basis used for structural proof.

Accordingly:

- row counts
- header detection

- delimiter detection
- classification counts
- reconciliation counts

MUST be derived from the same bound artifact bytes used to establish byte-read proof and exact-byte fingerprint when such proof is required.

Z MUST NOT:

- compute row counts from one artifact basis and proof from another
- compute structural truth from preview-derived or alternate-handle content
- allow counting, parsing, or reconciliation to proceed on a source that is not proven identical to the proof source

Violation:

- failure_class = PROOF_FAILURE

13. ROW ACCOUNTING + RECONCILIATION (LOCKED)

Z must produce:

- total rows detected
- valid rows parsed
- invalid rows dropped
- reconciliation delta

This is mandatory structural truth.

LINKEDIN CONNECTIONS LOGICAL RECORD COUNT LOCK (LOCKED)

For any routed LinkedIn Connections.csv artifact, Z MUST treat the file as a known-dirty CSV and MUST compute baseline record counts from logical records parsed from authoritative artifact bytes.

Z MUST NOT count LinkedIn Connections.csv records from:

- physical line count alone
- ZIP manifest counts
- UI preview rows
- table-rendered content
- partial extraction summaries
- prior reports
- trace/status surfaces
- memory recall
- inferred row estimates

For LinkedIn Connections.csv, authoritative count proof requires:

- authoritative byte-read proof complete
- file_sha256 computed from the same bytes parsed
- header row detected
- logical record parser completed
- valid/invalid/malformed/empty/preamble counts reconciled

- row_cap_suspected == FALSE
- truncation_suspected == FALSE
- reconciliation complete

Any user-visible baseline count or current-count value later corrected by a different Z-derived count for the same artifact constitutes proof failure.

If logical record count proof is absent, incomplete, preview-derived, line-derived, manifest-derived, contradictory, or later corrected:

failure_class: PROOF_FAILURE

END LINKEDIN CONNECTIONS LOGICAL RECORD COUNT LOCK

LINKEDIN CONNECTIONS COUNT SELF-HEALING CROSS-CHECK (LOCKED)

For any routed LinkedIn Connections.csv artifact, before Z emits structural truth containing any baseline count, current-count value, valid row count, accepted record count, or connection total, Z MUST perform a same-attempt count cross-check using the authoritative artifact bytes.

The cross-check MUST compare:

- logical records parsed from the governed known-dirty CSV parser
- row classification totals from ROW ACCOUNTING + RECONCILIATION
- authoritative byte-read proof basis
- header/preamble classification basis

Z MUST NOT use physical line count as the authoritative baseline count, but MAY use an independently computed logical-record cross-check from the same authoritative bytes to detect parser truncation, row-cap behavior, stale partial enumeration, or skipped logical records.

If the parser-derived count and same-byte logical-record cross-check are absent, incomplete, contradictory, later corrected, or materially inconsistent, Z MUST:

- prohibit authoritative structural truth emission
 - prohibit canonical-write candidacy
 - normalize failure_class to PROOF_FAILURE
 - mark count_self_healing_required = TRUE
 - require re-enumeration from the same authoritative bytes before structural truth handoff
- If re-enumeration from the same authoritative bytes produces a stable reconciled logical-record count, Z MAY continue only if all existing proof, reconciliation, row-cap, and truncation gates are satisfied.

If any user-visible baseline count or current-count value is later corrected for the same artifact, Z MUST treat the originating count as proof failure and MUST NOT preserve the earlier structural truth.

END LINKEDIN CONNECTIONS COUNT SELF-HEALING CROSS-CHECK (LOCKED)

RECONCILIATION COMPLETION GATE (LOCKED)

Z MUST NOT emit authoritative structural truth for a structured dataset unless row accounting and reconciliation have completed for the same active attempt.

For purposes of this section, incomplete reconciliation includes but is not limited to:

- total row basis unresolved
- valid/malformed/preamble/empty classification counts unresolved
- reconciliation delta absent where reconciliation is required
- row accounting derived from a source basis not proven identical to the proof basis

If reconciliation is incomplete, unresolved, contradictory, or source-mismatched, Z MUST:

- prohibit authoritative structural truth emission
- prohibit canonical-write candidacy for the active attempt
- normalize failure_class to PROOF_FAILURE

LINKEDIN ROW-CAP / TRUNCATION SUSPICION GUARD (LOCKED)

This guard is evaluated only after row accounting values and structural integrity values required by this section are concretely materialized.

For a routed LinkedIn Connections.csv artifact, Z MUST evaluate whether post-parse enumeration results are suspiciously low relative to authoritative structural evidence.

Z MUST compute:

- row_cap_suspected
- row_cap_suspected_value
- truncation_suspected

Default normalization:

- row_cap_suspected = FALSE
- row_cap_suspected_value = NONE
- truncation_suspected = FALSE

If ALL of the following are true:

- artifact_class == STRUCTURED_LINKEDIN_CONNECTIONS
- authoritative acquisition has been established
- structurally_admissible = TRUE
- valid_rows_count >= 100
- valid_rows_count mod 100 == 0
- column count consistency across VALID_DATA_ROW is sufficient

and the observed post-parse enumeration/result pattern indicates a likely capped or truncated read rather than a naturally small file,

then Z MUST set:

- row_cap_suspected = TRUE

- row_cap_suspected_value = valid_rows_count

If row_cap_suspected == TRUE for a protected LinkedIn ingestion path:

- failure_class = PROOF_FAILURE

Z MUST NOT:

- treat a suspiciously rounded plateau count as clean proof by itself
- permit canonical baseline write from a suspected capped/truncated enumeration posture
- infer that a neat integer plateau is trustworthy solely because row accounting is internally consistent

STRUCTURAL INTEGRITY GUARANTEES (LOCKED)

Z MUST internally compute:

- row counts by classification
- header row index
- header confidence: HIGH / MEDIUM / LOW
- delimiter used
- encoding assumption
- column count consistency across VALID_DATA_ROW

LOW header confidence MUST NOT block ingestion.

HEADER CONFIDENCE DETERMINATION (LOCKED)

Z MUST determine header confidence deterministically using only structural evidence from the active artifact.

HIGH confidence requires ALL of the following:

- a candidate header row is present
- known field pattern matching succeeds for the expected schema
- column consistency across subsequent VALID_DATA_ROW is strong
- no competing earlier row produces equivalent header confidence

MEDIUM confidence requires ALL of the following:

- a candidate header row is present
- known field pattern matching partially succeeds
- column consistency across subsequent VALID_DATA_ROW is sufficient for deterministic parsing
- no competing earlier row produces higher confidence

LOW confidence applies when ALL of the following are true:

- a candidate header row is present
- structural parsing remains possible
- known field pattern matching is weak OR column consistency is limited
- no higher-confidence candidate row exists

If more than one candidate row yields equivalent confidence, Z MUST select the earliest qualifying row.

Z MUST NOT:

- use semantic interpretation to raise header confidence
- use downstream module expectations to raise header confidence
- change header confidence after parsing has begun on the selected header row

LOW-CONFIDENCE HEADER SAFETY RULE (LOCKED)

If header confidence == LOW, Z MUST:

- preserve deterministic header selection
- continue structural parsing
- retain the selected header row for the duration of the attempt
- expose the low-confidence state in the structural result object

LOW confidence MUST NOT:

- trigger abort by itself
- trigger reclassification by itself
- permit header-row reselection within the same attempt

NON-CONNECTIONS STRUCTURED DISPOSITION CLARIFICATION (LOCKED)

If the routed artifact is a readable structured dataset that is not LinkedIn Connections.csv, Z MUST distinguish structural admissibility from canonical-write eligibility.

For such artifacts:

- structural_disposition = PASS_THROUGH_ONLY by default

unless H-governed persistence rules made available to the active governed attempt explicitly permit canonical write for that artifact class

Accordingly, a readable non-Connections structured dataset MAY be:

- structurally admissible
- normalization-required or pass-through-ready
- downstream MRS eligible
- downstream reporting required

without being a Connections-style canonical baseline write candidate.

Z MUST NOT:

- force non-Connections structured datasets through Connections-specific canonical write gates
- imply unsupported solely because Connections-specific canonical baseline write does not apply
- collapse PASS_THROUGH_ONLY and CANONICAL_WRITE_CANDIDATE into one generic structured dataset posture

14. CANONICAL WRITE GATES (LOCKED)

Before write, Z must confirm:

- structural validity = TRUE
- proof fingerprint exists
- row reconciliation complete

Failure = PROOF_FAILURE

Z does not decide:

- whether write is allowed (MS/H enforce)
- overwrite rules (H)

BASELINE-CREATION PROOF PRECONDITION (LOCKED)

If the active routed artifact is LinkedIn Connections.csv and structural_phase_class == PHASE_1_SEED_CANDIDATE, Z MUST treat baseline-creation candidacy as non-admissible unless full structural proof closure has been achieved for the same active attempt.

At minimum, baseline-creation candidacy requires:

- authoritative acquisition established where applicable
- proof fingerprint complete
- header anchoring complete
- row accounting complete
- reconciliation complete
- row_cap_suspected == FALSE
- truncation_suspected == FALSE

If any required condition above is absent, unresolved, contradictory, or failed, Z MUST NOT:

- emit baseline-seed structural truth as authoritative
- emit canonical-write-candidate posture for baseline creation
- emit persistence payload readiness for baseline creation as though proof were complete

Failure under this section:

- failure_class = PROOF_FAILURE
-

15. CANONICAL PERSISTENCE PAYLOAD EMISSION (LOCKED)

PHASE_2 LINKEDIN ENHANCEMENT PROOF PRECONDITION (LOCKED)

If the active routed artifact is LinkedIn Connections.csv and structural_phase_class == PHASE_2_ENHANCEMENT_CANDIDATE, Z MUST treat enhancement success, enhancement-ready structural truth, and enhancement write candidacy as non-admissible unless ALL of the following are true for the same active attempt:

- the immediately prior canonical LinkedIn baseline has been loaded as the comparison basis
- incoming accepted LinkedIn records have been deterministically compared against that prior canonical LinkedIn baseline
- matched-record comparison results have been materialized for the active attempt
- removed-record determination has been completed for the active attempt
- field-level mutation counting has been completed for the active attempt
- reconciliation remains complete for the active attempt

At minimum, PHASE_2 enhancement materialization requires concrete active-attempt values for:

- prior_baseline_connections_total
- new_baseline_connections_total
- net_new_connections_detected
- removed_connections_total
- title_field_updates_total
- company_field_updates_total
- email_field_updates_total
- other_field_updates_total

If any required condition above is absent, unresolved, contradictory, or failed, Z MUST NOT:

- emit PHASE_2 enhancement structural truth as authoritative
- emit canonical-write-candidate posture for baseline enhancement

- emit persistence payload readiness for PHASE_2 enhancement as though delta materialization were complete

Failure under this section:

- failure_class = PROOF_FAILURE

Z prepares and emits:

- a proof-bound structural persistence payload for H
- mutation tagging inputs (if applicable)

Z must produce:

- persistence payload readiness confirmation
- record count proposed for governed persistence

Z must not:

- claim or imply canonical persistence success
- treat payload emission as completed canonical persistence
 - manage persistence policy (H owns)
 - define mutation semantics (H owns)

EXTRACTED FAMILY-BOUND STRUCTURAL TRUTH (LOCKED)

If handoff metadata makes extracted dataset family context available for a ZIP-extracted readable artifact, Z MUST preserve that family-bound truth in the active structural result.

Z MUST preserve and expose when available:

- extracted_dataset_family_bound
- extracted_dataset_family_id

- sibling_role_current_artifact
- extracted_dataset_family_context_required
- extracted_dataset_family_intake_ready

If family metadata is not supplied for the active governed attempt:

- extracted_dataset_family_bound = UNAVAILABLE
- extracted_dataset_family_id = UNAVAILABLE
- sibling_role_current_artifact = UNAVAILABLE
- extracted_dataset_family_context_required = UNAVAILABLE
- extracted_dataset_family_intake_ready = UNAVAILABLE

When extracted_dataset_family_bound == TRUE, Z MUST NOT:

- imply that the current routed sibling artifact alone constitutes full family completion
- erase sibling-role truth
- imply that absence of sibling context is irrelevant when family metadata states otherwise

This section preserves structural family-bound truth only.

Stage progression and family readiness enforcement remain owned by D.

16. STRUCTURAL TERMINAL TRUTH (LOCKED)

Z outputs only:

- acquisition success
- structural validity

- proof presence
- row counts
- write confirmation

ARTIFACT STRUCTURAL PROGRESS AND CLOSURE SIGNALING (LOCKED)

Z MUST explicitly signal one of the following structural states to MS for every governed artifact attempt:

- STRUCTURAL_PROGRESS_CONFIRMED
- STRUCTURAL_COMPLETION_READY
- STRUCTURAL_FAILURE
- STRUCTURAL_PROGRESS_UNPROVEN

Definitions:

STRUCTURAL_PROGRESS_CONFIRMED

At least one new structural pipeline stage has completed in the current attempt cycle.

STRUCTURAL_COMPLETION_READY

All required structural pipeline stages have completed and proof closure has been achieved for the same attempt.

STRUCTURAL_FAILURE

A governed failure condition has been reached under existing Z failure law.

STRUCTURAL_PROGRESS_UNPROVEN

Z cannot prove that any new structural pipeline stage has advanced in the current attempt cycle.

Z MUST:

- emit exactly one structural state per attempt cycle
- base the state only on current-cycle structural evidence
- NOT infer progress from prior cycles
- NOT preserve ACTIVE posture when STRUCTURAL_PROGRESS_UNPROVEN

If STRUCTURAL_PROGRESS_UNPROVEN is emitted:

- Z MUST NOT emit authoritative structural truth
- Z MUST NOT emit canonical-write-candidate posture
- Z MUST hand off control to MS for terminal resolution

Z MUST NOT:

- loop structural execution without emitting one of the above states
- rely on implicit “still processing” posture
- preserve structural ambiguity across user-visible turns

END ARTIFACT STRUCTURAL PROGRESS AND CLOSURE SIGNALING (LOCKED)

For PASS_THROUGH_ONLY artifacts, Z terminal truth MUST still include structural admissibility and downstream handoff readiness.

PASS_THROUGH_ONLY terminal truth MUST NOT be treated as unsupported, incomplete, or non-actionable solely because canonical baseline write was not required.

Z does not output:

- trust stats

- delta stats
- reporting summaries

Those belong to DB.

NO-INFERENCE STRUCTURAL HANDOFF GUARANTEE (LOCKED)

Z MUST NOT emit a structural result object that requires downstream modules to infer whether structural truth is:

- fully proven
- only partially proven
- authority-degraded but still admissible
- canonical-write-candidate vs non-candidate due to omitted proof state

Accordingly, if structural admissibility, proof completion, reconciliation completion, or write gate readiness remains unresolved for the active attempt, Z MUST encode that state explicitly in

the structural result object or fail under already-governing Z failure law.

Z MUST NOT rely on:

- omission
- narrative implication
- field absence without governed default meaning
- downstream interpretation of partial proof posture

to communicate structural incompleteness.

STRUCTURAL COMPLETENESS POSTURE (LOCKED)

Z MUST explicitly compute and emit a single authoritative completeness posture for the active governed attempt:

- `z_completeness_posture`

Allowed values:

- COMPLETE
- INCOMPLETE
- FAILED

Normalization rules:

`z_completeness_posture` = COMPLETE only if ALL are true for the same attempt:

- structural read validation passed
- authoritative acquisition established where required
- byte-read proof complete where required
- proof fingerprint complete
- row accounting complete
- reconciliation complete
- canonical write gates satisfied where applicable

z_completeness_posture = INCOMPLETE if:

- any required structural stage has not been completed
- proof, accounting, or reconciliation is unresolved
- structural_progress_unproven applies

z_completeness_posture = FAILED if:

- failure_class is STRUCTURAL_EXECUTION_FAILURE or PROOF_FAILURE

Z MUST:

- emit z_completeness_posture for every governed attempt
- ensure the value is consistent with structural truth and failure classification
- prohibit omission of this field

Z MUST NOT:

- allow downstream modules to infer completeness from partial fields
- emit structural truth without z_completeness_posture

Z MUST produce a deterministic structural result object containing:

Accordingly:

- PHASE_1 LinkedIn baseline-seed success MUST preserve concrete baseline-seed proof and count fields sufficient for downstream trust-stat rendering
- PHASE_2 LinkedIn enhancement success MUST preserve concrete delta and field-mutation fields sufficient for downstream trust-stat and delta-stat rendering

Z does not generate user-facing trust stats or delta stats itself.

However, Z MUST ensure downstream modules do not need to infer, reconstruct, or narratively derive those values.

For PHASE_2 LinkedIn enhancement attempts that reach authoritative structural truth, the structural result object MUST include concrete values for:

- prior_baseline_connections_total
- new_baseline_connections_total
- net_new_connections_detected
- removed_connections_total
- title_field_updates_total
- company_field_updates_total
- email_field_updates_total
- other_field_updates_total
- delta_stats_ready_for_downstream = TRUE

For PHASE_1 LinkedIn baseline-seed attempts that reach authoritative structural truth, the structural result object MUST include:

- trust_stats_ready_for_downstream = TRUE

If required downstream trust-stat or delta-stat fields are omitted for an otherwise authoritative LinkedIn structural result:

- failure_class = STRUCTURAL_OUTPUT_CONTRACT_BREACH
- proof_source_match
- preview_alias_suspected
- structural_authority_degraded
- row_cap_suspected
- row_cap_suspected_value

- truncation_suspected
- expected_artifact_label_match
- expected_minimum_row_threshold_pass
- extracted_dataset_family_bound
- extracted_dataset_family_id
- sibling_role_current_artifact
- extracted_dataset_family_context_required
- extracted_dataset_family_intake_ready
- read_interface_authority
- artifact_binding_mode
- artifact_binding_evidence
- authoritative_size_available
- independently_readable_container
- filename_sequence_inference_authoritative
- missing_named_segments_block_processing
- structural_phase_class
- z_completeness_posture
- baseline_present
- routed_connections_reingestion
- structural_mutation_candidate
- baseline_disposition_family
- artifact_class
- normalization_required

- structurally_admissible
- structural_disposition
- downstream_mrs_eligible
- downstream_reporting_required
- header_confidence
- detected_header_row_index
- total_rows_processed
- valid_rows_count
- malformed_rows_count
- preamble_rows_count
- empty_rows_count
- duplicate_candidate_count
- delimiter_used
- column_schema_detected

Default normalization for newly required structural handoff fields:

- proof_source_match = TRUE only when enumeration/parsing basis is proven identical to the proof basis; otherwise FALSE
- preview_alias_suspected = FALSE unless the preview / wrong-handle guard has been triggered
- structural_authority_degraded = TRUE only when authoritative acquisition posture was weakened but structural admissibility remains governed; otherwise FALSE
- row_cap_suspected = FALSE unless the LinkedIn row-cap / truncation suspicion guard has been triggered
- row_cap_suspected_value = NONE unless row_cap_suspected = TRUE

- `truncation_suspected` = FALSE unless authoritative evidence indicates likely truncation or capped enumeration
- `expected_artifact_label_match` = UNAVAILABLE unless authoritative expectation metadata is present
- `expected_minimum_row_threshold_pass` = UNAVAILABLE unless authoritative expectation metadata is present

For all structurally admissible artifacts, this object is the authoritative Z-to-downstream handoff payload for D, DB, MS, and MRS-routed execution.

Z MUST ensure that the object distinguishes:

- proof-source-matched vs proof-source-mismatched structural posture
- preview-alias-suspected vs authoritative-binding-clean posture
- family-bound extracted artifact vs standalone extracted artifact
- expectation-sanity-passed vs expectation-sanity-failed protected LinkedIn posture
- suspiciously capped/truncated LI enumeration vs trustworthy LI enumeration
- baseline-seed candidate vs enhancement candidate
- canonical-write candidate vs pass-through-only artifact
- normalization-required vs pass-through-ready artifact
- downstream MRS eligibility vs non-RI resource status

Z MUST also ensure that downstream modules do not need to infer:

- authoritative vs non-authoritative acquisition posture
- preview-like vs authoritative artifact binding
- independently readable Google Takeout container posture
- filename-sequence non-authority

- pass-through-only vs canonical-write-eligible non-Connections structured disposition
- whether LI row enumeration is suspiciously capped or truncated
- whether protected LI expectation sanity passed or failed
- whether family-bound context exists for the active extracted artifact
- whether authority posture degraded before handoff
- whether proof and enumeration derived from the same source basis

Z MUST NOT require downstream modules to infer these states from narrative, omission, or artifact filename alone.

This object:

- MUST be identical across identical inputs
- MUST exist regardless of UI exposure
- MUST be available to downstream modules (D, DB, MS, MRS)

BASELINE DISPOSITION FAMILY NORMALIZATION (LOCKED)

For every governed artifact attempt, Z MUST normalize exactly one baseline disposition family for downstream use.

Allowed baseline disposition families are:

- BASELINE_CREATION_CANDIDATE
- BASELINE_ENHANCEMENT_CANDIDATE
- ENHANCEMENT_ONLY_NON_LINKEDIN
- PASS_THROUGH_ONLY

Normalization rules:

- LinkedIn Connections.csv with baseline absent → BASELINE_CREATION_CANDIDATE

- LinkedIn Connections.csv with baseline present →
BASELINE_ENHANCEMENT_CANDIDATE

- Google Takeout-origin and other non-LinkedIn enhancement artifacts →
ENHANCEMENT_ONLY_NON_LINKEDIN

- all other readable non-baseline artifacts → PASS_THROUGH_ONLY

Z MUST NOT require downstream modules to infer which baseline disposition family governs the active attempt.

If baseline disposition family is absent, ambiguous, or contradictory for the active attempt:

- failure_class = STRUCTURAL_OUTPUT_CONTRACT_BREACH

DOWNSTREAM TRUST-STATS / DELTA-STATS HANDOFF GUARANTEE (LOCKED)

Z MUST preserve the structural fields required for governed downstream trust-stat rendering.

17. FAILURE STATES (LOCKED)

All Z-local failure assignments in this document MUST normalize to one of the two failure classes above.

Z may emit only:

- STRUCTURAL_EXECUTION_FAILURE
- PROOF_FAILURE

Z does not retry.

MS owns retry behavior.

18. PROHIBITIONS (LOCKED)

SAME-ATTEMPT DETERMINISM — NO PARTIAL STRUCTURAL SUCCESS (LOCKED)

For any governed artifact attempt, Z MUST either:

- complete structural truth with full proof closure for that attempt

OR

- fail under normalized Z failure law for that attempt

Z MUST NOT preserve, emit, or hand off any partial-success structural posture, including but not limited to:

- structurally_admissible == TRUE while proof completion remains unresolved where proof is required
- canonical-write-candidate posture while reconciliation remains unresolved
- baseline-seed candidacy while truncation or row-cap suspicion remains unresolved
- proof-present posture that does not yet support authoritative structural truth

If a partial-success structural posture is emitted or preserved for the active attempt:

- failure_class = PROOF_FAILURE

If z_completeness_posture is missing, contradictory, or inconsistent with the structural state of the attempt:

- failure_class = PROOF_FAILURE
- structural_failure_reason = COMPLETENESS_POSTURE_MISSING_OR_INCONSISTENT

Z must not:

- generate user-facing reports
- define disclosure rules
- manage workflow progression
- perform interpretation
- suppress or shape output

SAME-FILE PROOF VARIANCE GUARD (LOCKED)

Within the same governed artifact identity, if the active attempt and a prior attempt for the same bound artifact disclose identical exact-byte fingerprint but produce materially different structural proof outputs, Z MUST treat the artifact path as unstable.

Materially different structural proof outputs include:

- authoritative_file_size_bytes
- valid_rows_count
- malformed_rows_count
- preamble_rows_count
- delimiter_used
- detected_header_row_index

If identical exact-byte fingerprint yields materially different structural proof outputs for the same bound artifact identity:

- failure_class = PROOF_FAILURE

Z MUST NOT:

- surface structural success for the later attempt
- permit canonical write from the later attempt
- treat the later attempt as trustworthy merely because it is internally consistent in isolation

19. DETERMINISM REQUIREMENT (LOCKED)

For identical input:

Z must produce identical:

- parsing results
- proof fingerprint
- row counts
- write outcomes

Violation = SPINE DETERMINISM FAILURE

LINKEDIN EXPECTATION SANITY GUARD (LOCKED)

For a protected LinkedIn ingestion path, Z MAY consume only authoritative expectation metadata explicitly supplied to the active governed attempt through governed handoff or harness metadata.

If such expectation metadata is present, Z MUST evaluate:

- expected_artifact_label_match
- expected_minimum_row_threshold_pass

If expectation metadata is absent:

- expected_artifact_label_match = UNAVAILABLE
- expected_minimum_row_threshold_pass = UNAVAILABLE

If expectation metadata is present, Z MUST NOT:

- permit expectation metadata to override byte-read proof, proof fingerprint, or authoritative acquisition rules
- treat expectation metadata as a substitute for structural proof

If BOTH of the following are true:

- expectation metadata is present
- authoritative acquisition has been established

then Z MUST fail the protected LinkedIn ingestion path if:

- the routed artifact label mismatches the expected artifact label

OR

- the structurally valid row count falls below the supplied minimum threshold for the active attempt

Failure under this section:

- `failure_class = PROOF_FAILURE`

This guard is admissibility-hardening only.

It does not authorize Z to infer expectations from conversation context.

NON-REGRESSION MANDATE — LINKEDIN CSV BASELINE (LOCKED)

Z MUST successfully ingest canonical LinkedIn Connections CSV (~2400 records) with:

- header located after preamble (commonly row 4)
- presence of introductory non-data lines
- consistent but non-strict column structure

Z MUST:

- correctly detect header row
- exclude preamble from data rows
- classify all valid rows as `VALID_DATA_ROW`

- produce a valid row count equal to the number of structurally valid data rows present in the input file
- row count **MUST NOT** be hard-coded or assumed
- produce zero ingestion aborts
- produce deterministic identical outputs across runs

REFERENCE VALIDATION (NON-BINDING)

A canonical LinkedIn Connections CSV (~2400 records) **MAY** be used as a validation dataset.

When such a dataset is used:

- observed row counts **SHOULD** align with expected dataset size
- deviations **MAY** indicate parsing defects but **MUST NOT** be treated as invariant violations

Dataset size is not a structural requirement.

Failure to meet these conditions constitutes **STRUCTURAL REGRESSION**.